

Documentação do Projeto - Listas Estáticas

Etapa 3 | Listas Estáticas

Na terceira etapa, o desafio do módulo solicita que você implemente alguns métodos utilizando Estruturas de Dados Estáticas. As duas principais tarefas desta etapa são:

- **Na classe que representa uma lista estática** (classe criada em aula), implemente um método **RECURSIVO** que recebe um elemento por parâmetro e retorna a quantidade de vezes que este elemento aparece na lista. Você deve fazer os métodos público e privado. Assinatura do método público:

Código

```
public int contaElementos(E el)
```

- Implemente, em uma classe chamada **Etapa3**, o seguinte método:

Código

```
public boolean checkBrackets(Stack s1)
```

- Esse método verifica se uma expressão matemática tem os parênteses agrupados de forma correta. Os critérios são:
 - (1) Se o número de parênteses à esquerda e à direita é igual;
 - (2) Se todo parêntese aberto é seguido, posteriormente, por um fechamento de parêntese.

Por exemplo, as expressões “**((A+B))**” e “**A+B(**” violam a regra (1), e as expressões “**)A+B(-C**” e “**(A+B))-(-C+D**” violam a regra (2).

Um exemplo de expressão correta seria: “**((A+B)-(-C+D))**”.

Arquivo de Teste - PrincipalEtapa3.java

Implemente, em uma classe chamada **PrincipalEtapa3.java** para validar os métodos implementados na **StaticList** e na **StaticStack**. Ele inclui dois testes principais:

- **Teste 1:** Contagem de elementos em uma lista estática utilizando recursão. Esse teste insere valores em uma lista estática e verifica quantas vezes um elemento específico aparece nela.
- **Teste 2:** Verificação de agrupamento de parênteses em uma expressão matemática. Esse teste utiliza uma pilha para verificar se a expressão tem seus parênteses agrupados corretamente.

A seguir, uma descrição dos dois testes:

Código - PrincipalEtapa3.java

```
// Teste 1: Contar elementos em uma lista estática usando recursão
System.out.println("Teste 1: Contagem de Elementos em uma Lista Estática");
StaticList lista = new StaticList<>(10);
lista.insert(1, 0);
lista.insert(2, 1);
lista.insert(3, 2);
lista.insert(1, 3);
lista.insert(2, 4);

int contagem = lista.contaElementos(1);
System.out.println("O elemento 1 aparece " + contagem + " vezes na lista.");
System.out.println("-----");

// Teste 2: Verificação de parênteses agrupados corretamente
System.out.println("Teste 2: Verificação de Parênteses Agrupados");
StaticStack pilha = new StaticStack<>(5);
pilha.push('(');
pilha.push('A');
pilha.push('+');
pilha.push('B');
pilha.push(')');

Etapa3 etapa3 = new Etapa3();
boolean resultado = etapa3.checkBrackets(pilha);

System.out.println("A expressão tem parênteses agrupados corretamente? " + resultado);
System.out.println("-----");
```

Exemplo de Saída - Teste

Abaixo, um exemplo de saída esperada ao rodar os testes:

=====

RELATÓRIO DE EXECUÇÃO

=====

Teste 1: Contagem de Elementos em uma Lista Estática
0 elemento 1 aparece 2 vezes na lista.

Teste 2: Verificação de Parênteses Agrupados
A expressão tem parênteses agrupados corretamente? true

Diagrama de Classes

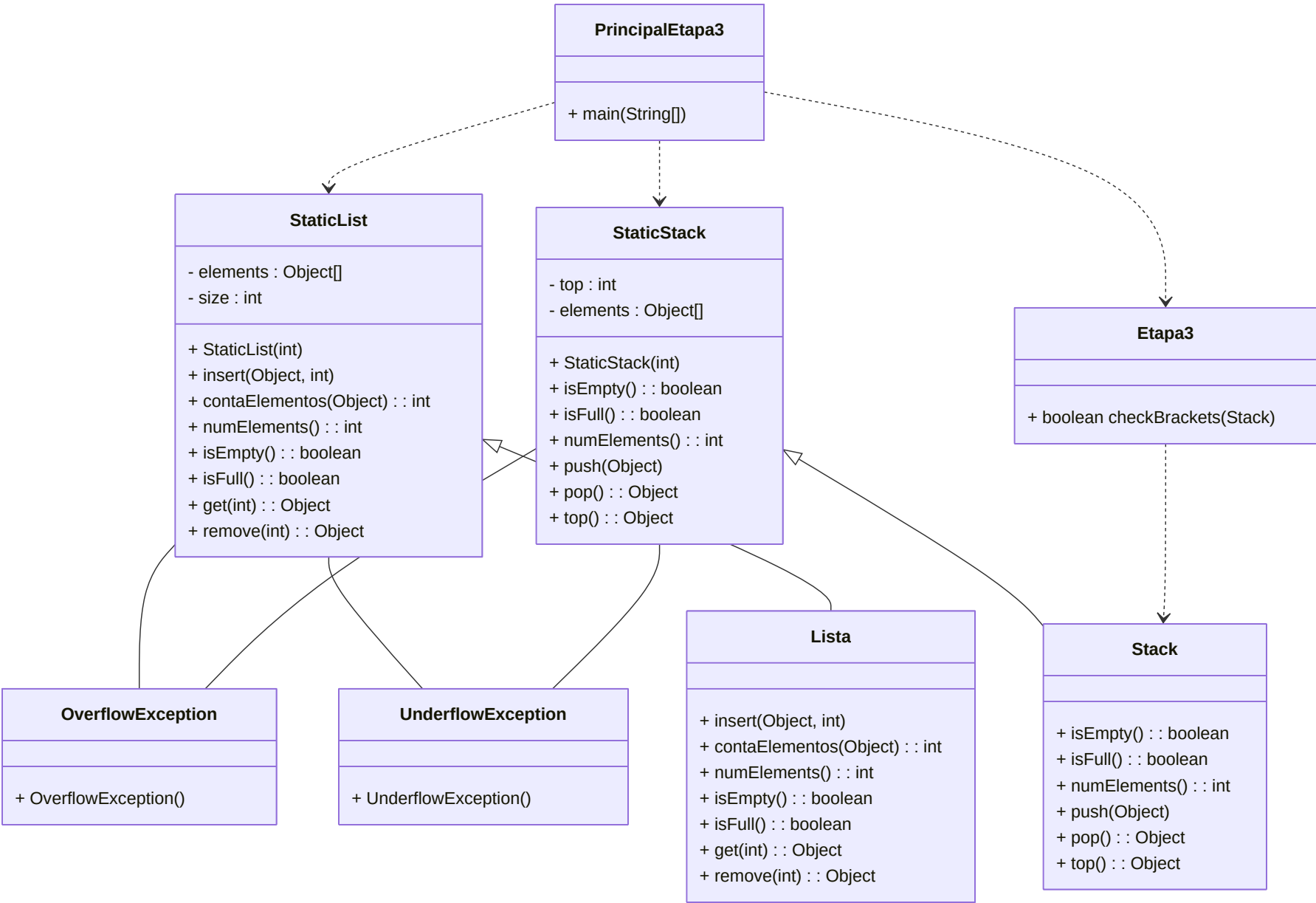
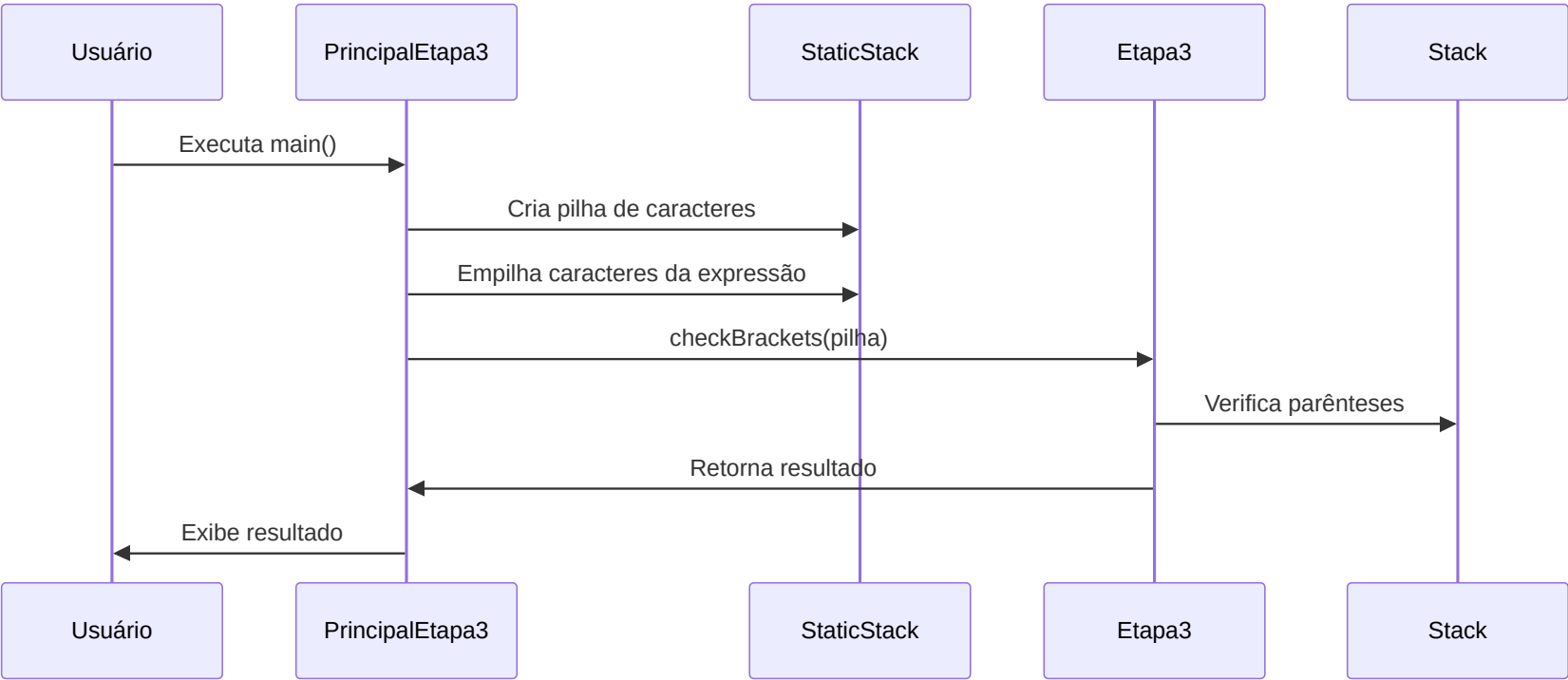


Diagrama de Sequência - Verificação de Parênteses



Descrição das Classes e Métodos

</> Classe PrincipalEtapa3 Descrição: Classe responsável por executar os testes das classes <code>StaticList</code> e <code>StaticStack</code>, contendo o método <code>main</code>. Método principal (main): - Executa dois testes: - Contar os elementos dentro de uma lista estática utilizando recursão. - Verificar o agrupamento de parênteses utilizando uma pilha estática.
☰ Interface Lista<E> Descrição: Interface que define os métodos principais para qualquer implementação de uma lista estática genérica, onde <code>(E)</code> representa o tipo de elementos que a lista pode armazenar. Métodos: <code>insert(E element, int index)</code>: Insere um elemento em uma posição específica da lista. <code>contaElementos(E el)</code>: Conta quantas vezes um determinado elemento aparece na lista. <code>numElements()</code>: Retorna o número de elementos atualmente na lista. <code>isEmpty()</code>: Verifica se a lista está vazia. <code>isFull()</code>: Verifica se a lista está cheia. <code>get(int index)</code>: Retorna o elemento em uma posição específica da lista. <code>remove(int index)</code>: Remove o elemento de uma posição específica e reorganiza os elementos restantes.
☰ Classe StaticList<E> Descrição: Implementa uma lista estática genérica com tamanho fixo para armazenar elementos de qualquer tipo <code>(E)</code>. Atributos: <code>elements : Object[]</code>: Array utilizado para armazenar os elementos da lista. <code>size : int</code>: Número de elementos atualmente presentes na lista. Métodos: <code>Construtor (StaticList(int maxSize))</code>: Inicializa a lista com um tamanho máximo definido pelo usuário. <code>insert(E element, int index)</code>: Insere um elemento em uma posição específica da lista. <code>contaElementos(E el)</code>: Conta quantas vezes um determinado elemento aparece na lista utilizando um método recursivo.

- `numElements()`: Retorna o número de elementos na lista.
- `isEmpty()`: Verifica se a lista está vazia.
- `isFull()`: Verifica se a lista está cheia.
- `get(int index)`: Retorna o elemento em uma posição específica.
- `remove(int index)`: Remove o elemento de uma posição específica e reorganiza os elementos restantes.

 Classe StaticStack<E>

Descrição: Implementa uma pilha estática genérica, onde os elementos seguem a regra *LIFO* (Last In, First Out).

Atributos:

- `top : int`: Aponta para o topo da pilha (último elemento inserido).
- `elements : Object[]`: Array utilizado para armazenar os elementos da pilha.

Métodos:

- `Construtor (StaticStack(int maxSize))`: Inicializa a pilha com um tamanho máximo definido pelo usuário.
- `isEmpty()`: Verifica se a pilha está vazia.
- `isFull()`: Verifica se a pilha está cheia.
- `numElements()`: Retorna o número de elementos na pilha.
- `push(E element)`: Adiciona um elemento ao topo da pilha.
- `pop()`: Remove e retorna o elemento do topo da pilha.
- `top()`: Retorna o elemento no topo da pilha sem removê-lo.
- `toString()`: Retorna uma representação em string da pilha.

 Classe Etapa3

Descrição: Classe que contém o método principal para verificar a correta organização dos parênteses em uma expressão matemática.

Método:

- `checkBrackets(Stack<Character> s1)`: Recebe uma pilha de caracteres que representam uma expressão matemática e verifica se os parênteses estão agrupados corretamente, respeitando as regras de abertura e fechamento.

 Classes de Exceções

OverflowException: Exceção lançada quando uma operação tenta adicionar um elemento a uma pilha ou lista que já está cheia.

UnderflowException: Exceção lançada quando uma operação tenta remover um elemento de uma pilha ou lista que está vazia.